A Mini Project Synopsis on

SmartClaim AI: Intelligent Vehicle Damage Detection & Insurance Claim Estimator

B.E. - I.T Engineering

Submitted By

Sunny Gupta 23104136

Pranav Ghodke 23104115

Aditya Chavan 23104090

Under The Guidance Of

Ms. Snehal Mali



DEPARTMENT OF INFORMATION TECHNOLOGY

A.P.SHAH INSTITUTE OF TECHNOLOGY

G.B. Road, Kasarvadavali, Thane (W), Mumbai-400615

UNIVERSITY OF MUMBAI

Academic year: 2026-27

CERTIFICATE

This is to certify that the Mini Project report on **SmartClaim AI : Intelligent Vehicle Damage Detection & Insurance Claim Estimator** has been submitted by Sunny Gupta (23104136), Pranav Ghodke (23104115) and Aditya Chavan (23104090) who are a Bonafide students of A. P. Shah Institute of Technology, Thane, Mumbai, as a partial fulfilment of the requirement for the degree in **Information Technology**, during the academic year **2026-2027** in the satisfactory manner as per the curriculum laid down by University of Mumbai.

Ms. Snehal Mali

Guide

External Examiner(s)

1.

Place: A.P. Shah Institute of Technology, Thane

Date:

ACKNOWLEDGEMENT

This project would not have come to fruition without the invaluable help of our guide **Ms. Snehal Mali**. Expressing gratitude towards our HoD, **Dr. Kiran Deshpande**, and the Department of Information Technology for providing us with the opportunity as well as the support required to pursue this project.

ABSTRACT

Motor insurance claims in India still depend on a surveyor visiting the vehicle, inspecting the damage and preparing a repair estimate by hand. The process is slow, varies from surveyor to surveyor, and gives the vehicle owner little idea of what the claim is worth until days later. **SmartClaim AI** is an AI assistant that turns a single photograph of a damaged vehicle, together with a few vehicle and policy details, into an instant, itemised repair and claim estimate.

The system combines three deep-learning models with a transparent pricing engine. A YOLOv8 classification model identifies where the damage is (front or rear) and how severe it is (normal, breakage, crushed), reaching **80.35% validation accuracy** and outperforming a custom CNN, EfficientNet-B0 and ResNet50 trained on the same 2,300-image dataset. Two YOLO11 instance-segmentation models then locate 21 car parts (mask mAP@0.5 of **0.81**) and eight damage types; each damage region is attributed to the part it overlaps, so the estimate names the actual parts to repair or replace.

A rule-based estimator prices every part using segment-wise costs, city-tier labour rates, paint and GST, and then applies Indian motor-insurance rules — the IRDAI depreciation schedule, compulsory and voluntary deductibles, the zero-depreciation add-on and the 75% constructive total-loss threshold — to split the bill into the amount the insurer pays and the amount the customer pays. An analysis of a public claims dataset showed that tabular claim records cannot predict repair cost from damage information, which motivated this part-level design. The complete pipeline runs on an Apple Silicon GPU and is delivered through a web application that returns the annotated photo and a low-to-high estimate in about two seconds.

Keywords: vehicle damage detection, insurance claim estimation, YOLO, instance segmentation, transfer learning, IRDAI depreciation, computer vision

TABLE OF CONTENTS

1. Introduction....1

1.1 Purpose....1

1.2 Objectives....1

1.3 Scope....2

2. Literature Survey....3

3. Problem Definition....4

4. Proposed System....6

4.1 System Architecture....6

4.2 Workflow....7

4.3 Algorithm....8

4.3.1 Algorithm 1: Damage Location and Severity Classification....8

4.3.2 Algorithm 2: Damage-to-Part Attribution....9

4.3.3 Algorithm 3: Repair Estimate and Claim Settlement....9

5. Technology Stack....11

5.1 Why YOLO Models....11

5.2 Why a Rule-Based Estimator....12

6. Implementation....13

6.1 Datasets....13

6.1.1 Damage Location and Severity Dataset....13

6.1.2 Car Parts and Damage Segmentation Dataset....14

6.2 Development Environment and GPU Acceleration....14

6.3 Model Training....15

6.3.1 Location and Severity Classifiers....15

6.3.2 Part and Damage Segmentation Models....16

6.4 Damage-to-Part Attribution....16

6.5 Pricing Engine and Policy Rules....16

6.6 Web Application....17

6.7 Testing 18

7. Results 19

7.1 Location and Severity Classification 19

7.2 Part and Damage Segmentation 21

7.3 End-to-End Claim Estimates 24

7.4 Performance 27

8. Conclusion 28

9. Future Scope 29

References 30

1. Introduction

1.1 Purpose

After a road accident, the first question a vehicle owner asks is how much the repair will cost and how much of it the insurance company will pay. Today that answer comes only after a claim is registered, a surveyor is assigned, the vehicle is inspected in person and a workshop prepares an estimate. Each step adds days, and the result depends heavily on the experience of the individual surveyor. Insurers, in turn, spend a large share of their claim-handling cost on inspecting small claims that could be assessed from photographs.

The purpose of **SmartClaim AI** is to show that this first assessment can be automated end to end. The user uploads a photograph of the damaged vehicle and enters a few details that every policy document already contains — the type of car, its age, engine capacity, Insured Declared Value (IDV), city and whether the policy has a zero-depreciation add-on. Within about two seconds the system returns:

- an annotated photograph showing the detected car parts and damage regions,
- the location and severity of the damage with the model's confidence,
- an itemised repair plan listing each part and whether it should be repaired, repainted or replaced,
- a low-to-high repair estimate in rupees including labour, paint and GST, and
- the claim settlement: how much the insurer pays after depreciation and deductible, and how much the customer pays.

Every rupee in the estimate can be traced back to a detected part and a documented rule.

1.2 Objectives

1. To build and compare deep-learning models that classify damage location (front/rear) and severity (normal, breakage, crushed).
2. To segment which car part is damaged and what type of damage it has.
3. To attribute each damage region to the car part it lies on.

4. To design a transparent pricing engine using part prices, car segment, labour rates, paint and GST.
5. To apply IRDAI rules: depreciation, deductibles, zero-dep cover and total loss.
6. To deliver the complete pipeline as an easy-to-use web application.

1.3 Scope

The scope of SmartClaim AI is as follows:

- Can be used to assess damage to passenger cars (hatchback, sedan, SUV and luxury segments) from a single photograph of the front or rear of the vehicle.
- Can be used to classify damage into six classes: front and rear × breakage, crushed and normal.
- Can be used to identify 21 exterior car parts such as bumpers, bonnet, boot lid, doors, fenders, lamps, glass, mirrors, wheels and licence plate.
- Can be used to detect eight damage types: dent, scratch, broken part, missing part, crack, paint chip, flaking and corrosion.
- Can be used to generate an itemised repair estimate from a catalogue of 23 priced parts with segment-wise prices and city-tier labour rates.
- Can be used to compute the claim settlement as per the IRDAI depreciation schedule, deductibles, zero-depreciation cover and the constructive total-loss rule.
- Can be used by vehicle owners and claim handlers through a browser-based web application to get an instant first estimate.
- Can be extended with multi-photo assessment, live part prices, fraud screening and a mobile application.

The estimate produced by SmartClaim AI is an indicative first assessment that helps the customer and the insurer start a claim with a shared, explainable number; the final settlement remains subject to the insurer's survey as required by regulation.

2. Literature Survey

The works in Table 2.1 shaped the design choices of SmartClaim AI.

Table 2.1: Literature survey summary

Sr. No.	Title	Authors	Findings and relevance to SmartClaim AI
1	Deep learning-based car damage classification (ICMLA 2017)	K. Patil, M. Kulkarni, A. Sriraman, S. Karande	Showed that CNNs can classify car damage even with roughly a thousand training images when transfer learning and data augmentation are used. We follow the same strategy: pretrained backbones with flips, rotation and colour jitter.
2	Deep Residual Learning for Image Recognition (CVPR 2016)	K. He, X. Zhang, S. Ren, J. Sun	Residual connections make very deep networks trainable. ResNet50 with its last block fine-tuned is one of the five classifiers we compared (73.9% accuracy).
3	EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks (ICML 2019)	M. Tan, Q. V. Le	Compound scaling of depth, width and resolution gives accurate, compact models. EfficientNet-B0 served as a frozen-feature baseline (60.7%).
4	You Only Look Once: Unified, Real-Time Object Detection (CVPR 2016)	J. Redmon, S. Divvala, R. Girshick, A. Farhadi	Introduced single-stage detection that predicts boxes and classes in one pass. The YOLO family (YOLOv8, YOLO11) powers all three SmartClaim AI models and keeps inference to about two seconds.
5	Mask R-CNN (ICCV 2017)	K. He, G. Gkioxari, P. Dollár, R. Girshick	Extended detection with a per-object segmentation mask. Pixel-level masks are what allow SmartClaim AI to measure how much of a part a damage covers and to attribute damage to parts.
6	Vehicle Damage Classification and Fraudulent Image Detection Including Moiré Effect Using Deep Learning (IEEE CCECE 2020)	U. Waqas, N. Akram et al.	Applied deep learning to insurance claim photos, combining damage classification with detection of fraudulent re-photographed images, highlighting the value of automated photo-based claim screening.
7	CarDD: A New Dataset for Vision-based Car Damage Detection (IEEE T-ITS 2023)	X. Wang, W. Li, Z. Wu	Released 4,000 high-resolution images with over 9,000 annotated instances of six damage types and benchmarked detection and segmentation models, establishing fine-grained damage segmentation as a practical task.

Gap identified. Most published work stops at classifying or detecting damage. None of the surveyed systems converts the visual result into a priced, itemised repair plan, and none applies the depreciation and deductible rules that decide what an Indian policyholder is actually paid. SmartClaim AI closes this gap by combining part-level segmentation with an explainable, regulation-aware estimator.

3. Problem Definition

Motor own-damage claims follow a manual, inspection-driven process. For the customer the claim amount is unknown until a surveyor visits; for the insurer every claim, however small, needs a person on site. The specific problems addressed by this project are:

1. **Slow first assessment:** the customer waits for an inspection before learning whether a claim is worth filing.
2. **Inconsistent estimates:** two surveyors can value the same damage differently because there is no shared, explainable method.
3. **Opaque settlements:** customers rarely understand why the amount paid is lower than the garage bill — depreciation and deductibles are applied out of sight.
4. **Damage alone does not decide cost:** the same dent costs very different amounts on a hatchback bumper and on a luxury SUV door, so a useful estimate must know the part, the vehicle segment and the policy terms.
5. **Existing claim data cannot be reused for pricing:** public insurance datasets record claim amounts, but not in a form that links them to the visible damage.

The last point was verified experimentally. A widely used public auto-insurance claims dataset (1,000 claims, 40 attributes including incident severity, collision type, vehicle make and claim amounts) was analysed to see whether a regression model could predict the vehicle repair claim from damage information. A gradient-boosting model reached an apparent R^2 of 0.69, but this came entirely from separating collisions from theft and parked-car claims. Restricted to collisions — the only claims a damage photo can describe — the cross-validated R^2 fell to **−0.14**, i.e. worse than predicting the average. As Figure 3.1 shows, minor damage, major damage and total loss all average about $45,000 in that dataset.

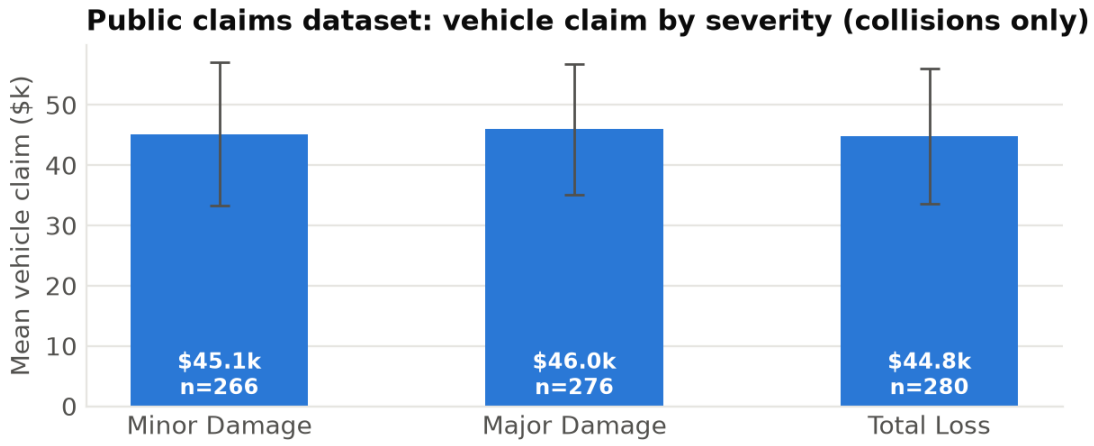


Figure 3.1: Vehicle claim by incident severity in a public claims dataset (collisions only); error bars show one standard deviation

Problem statement. Design and implement a system that takes a photograph of a damaged car and basic vehicle and policy details, identifies the damaged parts and the nature of the damage, and produces an itemised, explainable repair estimate and claim settlement consistent with Indian motor-insurance rules — without relying on historical claim amounts that do not reflect the visible damage.

4. Proposed System

SmartClaim AI is organised as a layered system in which deep-learning models produce visual evidence and a transparent rule engine converts that evidence into money. Keeping the two concerns separate means every model can be retrained without touching the pricing rules, and every price can be changed without retraining a model.

4.1 System Architecture

Figure 4.1 shows the components of the system, organised in five layers, with the offline model-training pipeline alongside:

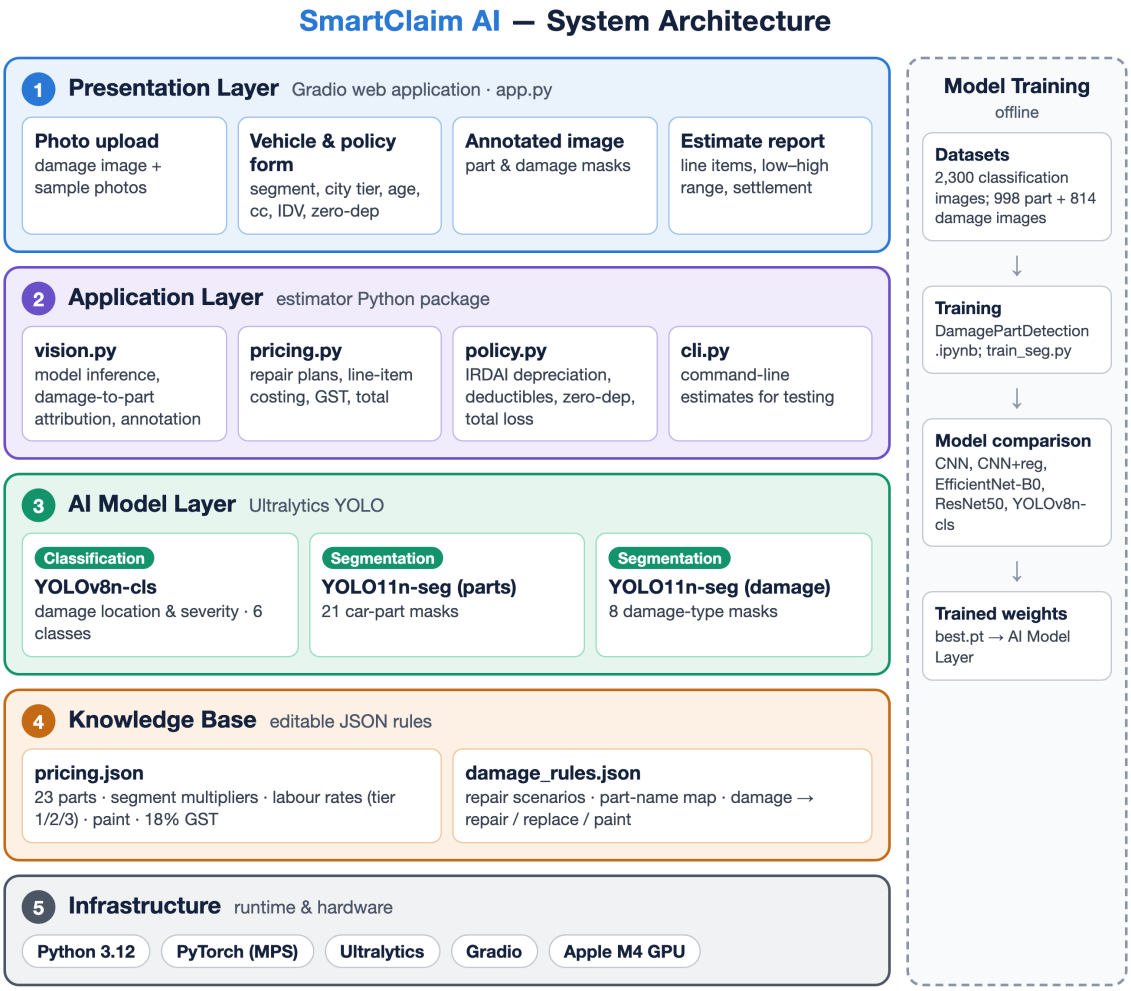


Figure 4.1: System architecture of SmartClaim AI

- **Presentation layer:** the Gradio web application (*app.py*) with a photo upload, a vehicle and policy form, the annotated result image and the estimate report.

- **Application layer:** the *estimator* Python package. *vision.py* runs the models and attributes damage to parts; *pricing.py* builds repair plans and costs each line item; *policy.py* encodes the IRDAI depreciation schedule, deductibles, zero-depreciation cover and the total-loss rule; *cli.py* produces estimates from the command line.
- **AI model layer:** three Ultralytics YOLO models — YOLOv8n-cls for damage location and severity, and two YOLO11n-seg models for 21 car parts and 8 damage types.
- **Knowledge base:** two editable JSON files. *pricing.json* holds the part catalogue, segment multipliers, labour rates, paint cost and GST; *damage_rules.json* holds the repair scenarios, the part-name mapping and the rules that turn a damage type into a repair action.
- **Infrastructure:** Python 3.12, PyTorch with the Metal Performance Shaders (MPS) backend, Ultralytics and Gradio, running on the Apple M4 GPU.
- **Model training (offline):** the datasets are used to train and compare the models in the training notebook and scripts; the resulting weights are loaded by the AI model layer.

4.2 Workflow

Figure 4.2 traces a single request through the system, from the uploaded photo to the settlement shown to the user. The two decision points that matter most are whether the damage model finds any damage (if not, the classifier's assessment is used on its own) and whether the repair bill exceeds 75% of the IDV, in which case the claim is settled as a constructive total loss.

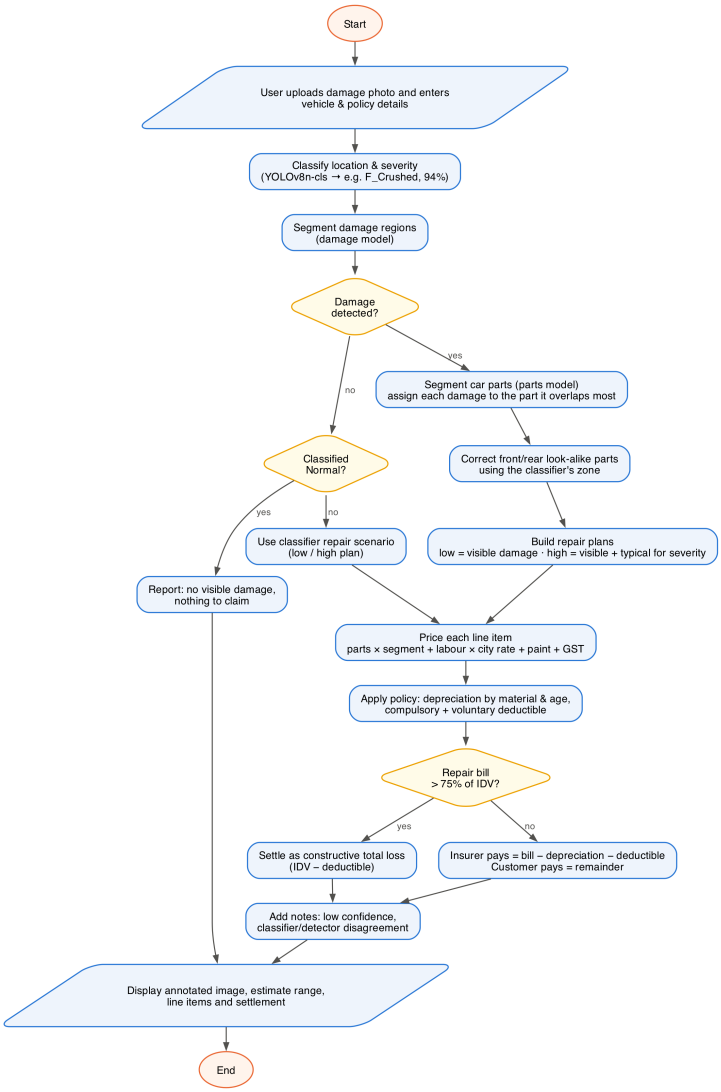


Figure 4.2: End-to-end workflow of a claim estimate

4.3 Algorithm

The system is driven by three algorithms that run in sequence for every photograph.

4.3.1 Algorithm 1: Damage Location and Severity Classification

Input: RGB photograph I. **Output:** label $y \in \{F, R\} \times \{Breakage, Crushed, Normal\}$ and confidence c.

1. Load the YOLOv8n-cls weights once and keep them on the GPU (MPS) for all later requests.
2. Convert the image to RGB (photos with an alpha channel are flattened) and resize it to 224 × 224 pixels.

3. Normalise pixel values and pass the tensor through the network to obtain six class scores z.
4. Apply the softmax function: $p(k) = \exp(z_k) / \Sigma_j \exp(z_j)$.
5. Select $y = \text{argmax}_k\, p(k)$ and $c = \max_k p(k)$; split y into the zone (F or R) and the severity level.
6. If $c < 0.5$, attach a *low-confidence* note recommending manual inspection.

4.3.2 Algorithm 2: Damage-to-Part Attribution

Input: image I, zone from Algorithm 1. **Output:** list of detections (damage type, part, area fraction, part fraction, confidence).

1. Run the damage segmentation model with confidence threshold 0.3 to obtain damage masks $D_1 \ldots D_n$ and their classes.
2. Run the car-part segmentation model on the same image to obtain part masks $P_1 \ldots P_m$ at the original resolution.
3. For every damage mask D_i compute the overlap with each part: $\text{overlap}(i, j) = |D_i \cap P_j| / |D_i|$.
4. Assign D_i to the part j *with the largest overlap if* $\text{overlap}(i, j) \geq 0.2$; otherwise fall back to a default part for that damage type in the classified zone (for example a crack in the front zone → front bumper).
5. If the part is a front/rear look-alike (bumper, lamp, bonnet/boot lid, windscreen) and disagrees with the classifier's zone, replace it with its counterpart in the classified zone.
6. Record the area fraction $|D_i| / |I|$ and the part fraction $|D_i| / |P_j^*|$, used later to decide repair versus replacement.

4.3.3 Algorithm 3: Repair Estimate and Claim Settlement

Input: classifier label, detections, vehicle (segment, city tier) and policy (age, engine cc, IDV, zero-dep, voluntary deductible). **Output:** low and high scenarios, each with line items and settlement.

1. **Build plans.** For each detection, look up the [low, high] repair action of its damage type (e.g. dent → [repair, repair], broken part → [replace, replace], scratch → [paint, paint]). A dent covering more than 30% of its part is upgraded to *replace* in the high plan. The **low plan** contains only visible damage; the **high plan** adds the classifier's typical repair scenario for the detected severity, covering damage hidden behind the panels. When no damage is detected the classifier's scenarios are used directly. Duplicate parts keep the most extensive action.
2. **Price each line item** using the part catalogue (s = segment multiplier, r = city labour rate per hour):

$$parts = price(part, action) \times s \qquad labour = hours(part, action) \times r$$

$$paint = 2{,}800 \times s \quad (painted\ panels\ only)$$

$$item = (parts + labour + paint) \times 1.18 \qquad (GST\ at\ 18\%)$$

3. **Depreciation** applies to replaced parts by material — rubber, nylon and plastic 50%, fibreglass 30%, glass 0%, metal 0–50% by vehicle age — and 50% on the material share (25%) of paint. With a zero-depreciation add-on it is zero.
4. **Deductible** = ₹1,000 for engines up to 1,500 cc or ₹2,000 above, plus any voluntary deductible.
5. **Settlement:** if the bill exceeds 75% of the IDV, the vehicle is a constructive total loss and the insurer pays IDV − deductible; otherwise insurer pays = bill − depreciation − deductible. The customer pays the remainder.

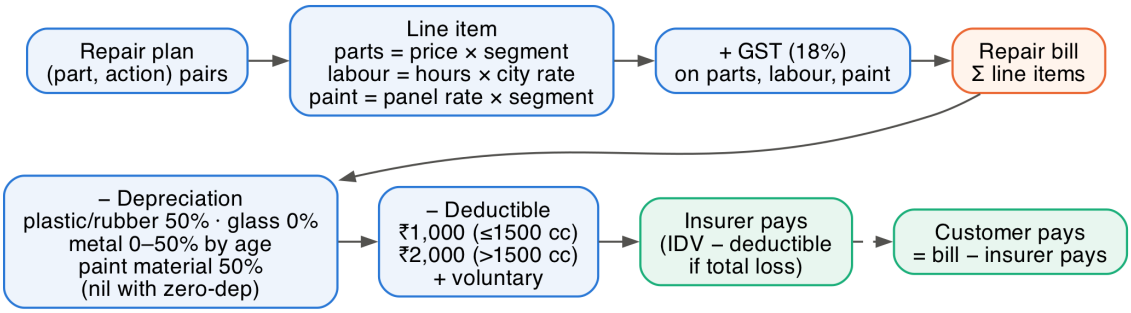


Figure 4.3: From repair plan to settlement (Algorithm 3)

5. Technology Stack

SmartClaim AI is written entirely in Python. Table 5.1 lists the technologies used in each part of the system.

Table 5.1: Technology stack

Layer	Technology	Purpose in SmartClaim AI
Deep learning	PyTorch 2.14, torchvision 0.29	Model definition, transfer learning (ResNet50, EfficientNet-B0) and GPU training
Detection & segmentation	Ultralytics 8.4 (YOLOv8n-cls, YOLO11n-seg)	Training and inference of the three production models; ONNX export
GPU acceleration	Apple Metal Performance Shaders (MPS)	Training and inference on the Apple M4 GPU through PyTorch
Data processing	NumPy, pandas, Pillow	Image loading, RGBA→RGB conversion, dataset statistics, results analysis
Machine learning	scikit-learn, XGBoost	Evaluation metrics, classification reports, regression analysis of claim data
Pricing engine	Python dataclasses + JSON	Repair plans, line-item costing, IRDAI depreciation and settlement rules
Web application	Gradio 6.30	Photo upload, vehicle/policy form, annotated image and estimate display
Visualisation	Matplotlib, seaborn, Graphviz	Training curves, confusion matrices, charts and diagrams
Testing	pytest	32 automated tests covering depreciation, deductibles, plans and settlement
Environment	Python 3.12, uv, Jupyter, Git	Reproducible virtual environment, notebooks and version control
Hardware	Apple M4 (10-core CPU, 10-core GPU), 16 GB unified memory, macOS 26	Development, training and deployment

5.1 Why YOLO Models

YOLO models were chosen for all three vision tasks because they offer pretrained weights for classification and segmentation, train quickly on small datasets, run in real time and share a single API for training, validation, inference and export. In our comparison the 1.4-million-parameter YOLOv8n-cls classifier was both the smallest and the most accurate model, and the nano segmentation models keep the full three-model pipeline at about two seconds per photo on a laptop GPU.

5.2 Why a Rule-Based Estimator

Repair pricing is governed by part prices, labour rates and published insurance rules rather than by patterns that a model must discover. Encoding them as data (JSON) and code makes every figure in the estimate explainable and auditable, lets an insurer update prices without retraining anything, and avoids learning from claim histories which, as shown in Chapter 3, do not reflect the visible damage.

6. Implementation

6.1 Datasets

6.1.1 Damage Location and Severity Dataset

The classifier was trained on 2,300 photographs of cars sorted into six folders by damage location and severity. Every image was verified to decode correctly; 310 images stored with an alpha channel are converted to RGB during loading. Table 6.1 gives the class distribution and Figure 6.1 shows one example per class.

Table 6.1: Damage location and severity dataset

Class	Meaning	Images
F_Breakage	Front, broken parts	500
F_Crushed	Front, crushed / deformed	400
F_Normal	Front, no damage	500
R_Breakage	Rear, broken parts	300
R_Crushed	Rear, crushed / deformed	300
R_Normal	Rear, no damage	300
Total	**75% training (1,725) / 25% validation (575)**	**2,300**



Figure 6.1: Sample images from the six damage location and severity classes

6.1.2 Car Parts and Damage Segmentation Dataset

Part and damage segmentation use the open *Car Parts and Damages* dataset (MIT licence), which provides polygon annotations in Supervisely format: 998 images labelled with 21 car parts (15,767 polygons) and 814 images labelled with eight damage types (9,084 polygons). A converter written for this project (*prepare_supervisely.py*) normalises class names, converts every polygon to the YOLO segmentation format and creates a reproducible 85/15 train–validation split.

Table 6.2: Segmentation datasets after conversion

Dataset	Train	Validation	Classes (instances)
Car parts	849	149	21 classes, e.g. back window 1,237 · mirror 987 · front window 951 · fender 935 · front door 917 · headlight 914 · … · back windshield 271
Damage	692	122	scratch 3,242 · dent 1,664 · broken part 1,500 · paint chip 1,356 · missing part 632 · flaking 337 · corrosion 277 · cracked 76

6.2 Development Environment and GPU Acceleration

All training and inference run on an Apple M4 laptop with 16 GB of unified memory. The original notebook code targeted NVIDIA CUDA only; it was adapted to select the best available device automatically:

```
if torch.cuda.is_available():
    device = torch.device('cuda')
elif torch.backends.mps.is_available():
    device = torch.device('mps')   # Apple Silicon GPU
else:
    device = torch.device('cpu')
```

Data loading was parallelised with four worker processes because JPEG decoding of large photographs, not the GPU, was the bottleneck. A profiling run of ResNet50 confirmed that a batch of 32 images at 224 × 224 uses only about 1.3 GB of GPU memory, so batch 32 was kept for classification and batch 16 at 640 × 640 for segmentation.

6.3 Model Training

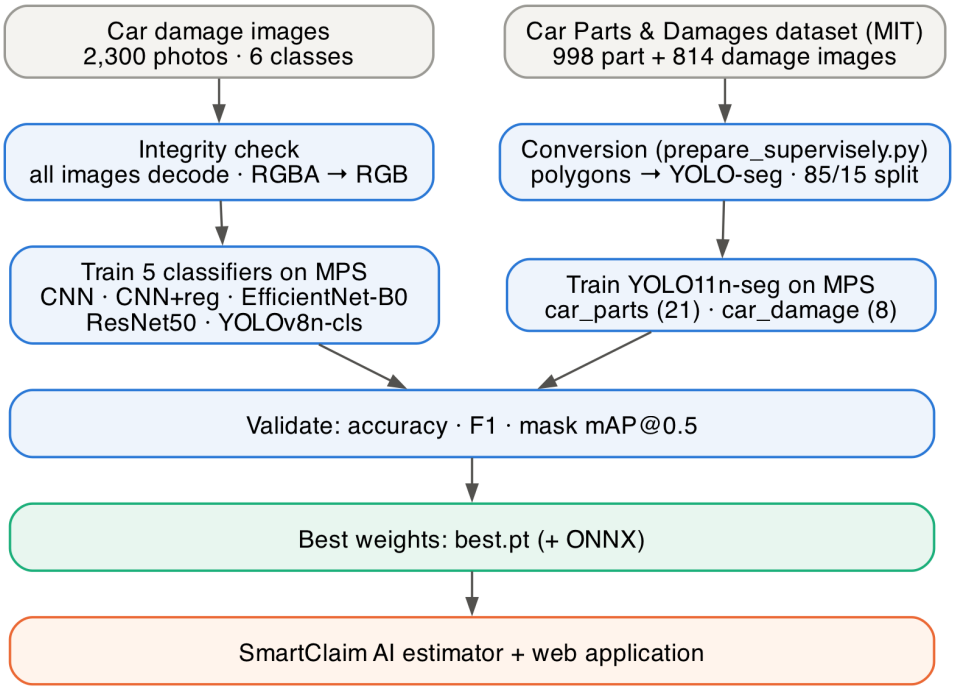


Figure 6.2: Data preparation and model training pipeline

6.3.1 Location and Severity Classifiers

Five classifiers were trained on the same split to choose the best architecture. All used 224 × 224 inputs, ImageNet normalisation and augmentation with horizontal flips, occasional vertical flips (p = 0.1), ±15° rotation and colour jitter. Table 6.3 lists their configurations.

Table 6.3: Classifier training configuration

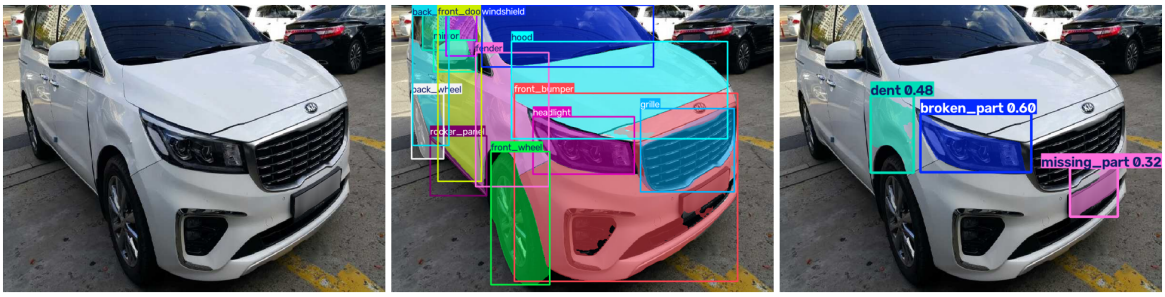
Model	Approach	Optimiser / LR	Epochs	Training time
Baseline CNN	3 conv blocks (16-32-64) + 512-unit FC	Adam, 1e-3	6	39 s
CNN + Regularisation	Baseline + batch norm, dropout 0.5	Adam, 1e-3, wd 1e-4	5	35 s
EfficientNet-B0	ImageNet backbone frozen, new classifier	Adam, 1e-3	5	55 s
ResNet50	ImageNet backbone, layer4 + FC trainable, dropout 0.39	Adam, 8.7e-4	10	275 s
YOLOv8n-cls	ImageNet-pretrained YOLOv8 nano classifier, batch 32	Ultralytics default	10	3.2 min

6.3.2 Part and Damage Segmentation Models

Both segmentation models start from COCO-pretrained YOLO11n-seg weights and were trained at 640 × 640 with batch 16 on the MPS GPU. The part model was trained for 6 epochs (about 1.75 minutes per epoch) and the damage model for 15 epochs; Ultralytics saves the best checkpoint by validation mask mAP after every epoch. The training script (*train_seg.py*) is generic, so either model can be retrained with more epochs or a larger backbone through command-line options.

6.4 Damage-to-Part Attribution

Figure 6.3 illustrates Algorithm 2 on a held-out photograph. The part model outlines the bumper, grille, headlight, bonnet, fender, doors and wheels; the damage model finds a broken headlight, a dent on the fender and a missing section of the front bumper. Because both models run at the original image resolution, their masks can be intersected directly:



(a) Input photo (b) Car-part segmentation (c) Damage segmentation

Figure 6.3: Damage-to-part attribution on a held-out photograph

```
overlaps = (part_masks & damage_mask).sum(dim=(1, 2)) / damage_area
best = overlaps.argmax()
if overlaps[best] >= MIN_PART_OVERLAP:          # 0.2
    part = PART_CLASSES[parts.names[best]]   # headlight -> headlamp
    part_fraction = damage_area / part_area[best]
```

6.5 Pricing Engine and Policy Rules

The pricing engine is data-driven. *pricing.json* holds a catalogue of 23 parts, each with a material, a replace price, an optional repair price, labour hours and whether it is a painted panel. Prices are stored for a hatchback and scaled by a segment multiplier (hatchback 1.0, sedan 1.35, SUV 1.8, luxury 4.0); labour is charged at ₹650, ₹500 or ₹400 per hour for tier-1, tier-2 and tier-3 cities. Table 6.4 shows an excerpt.

Table 6.4: Excerpt of the part catalogue (hatchback base prices)

Part	Material	Replace (₹)	Repair (₹)	Labour hrs (replace / repair)	Painted
Front / rear bumper	Plastic	4,500	1,500	1.5 / 2.0	Yes
Headlamp	Plastic	4,000	—	0.7	No
Bonnet	Metal	7,500	denting only	1.0 / 4.0	Yes
Front door	Metal	12,000	denting only	3.0 / 4.0	Yes
Windshield	Glass	6,500	—	1.5	No
Radiator	Metal	6,000	—	2.0	No
Tyre	Rubber	4,000	300	0.3 / 0.3	No

policy.py implements the depreciation schedule of the India Motor Tariff (Table 6.5), the compulsory deductible by engine capacity, the zero-depreciation add-on and the 75% constructive total-loss rule.

Table 6.5: Depreciation applied to replaced parts

Material / vehicle age	Depreciation
Rubber, nylon, plastic parts, tyres, tubes, batteries	50%
Fibreglass components	30%
Glass	Nil
Painting (on the material share of paint cost)	50%
Metal parts — vehicle age ≤ 6 months / 6–12 months	Nil / 5%
Metal parts — 1–2 years / 2–3 years	10% / 15%
Metal parts — 3–4 years / 4–5 years	25% / 35%
Metal parts — 5–10 years / above 10 years	40% / 50%

damage_rules.json maps each damage type to a repair action for the low and high plans, maps the 21 part classes to priced parts, and stores the classifier's typical repair scenario for each of the four damaged classes. Paint actions on unpainted parts such as glass and lamps are treated as cosmetic and are not billed.

6.6 Web Application

The user interface is built with Gradio (Figure 6.4). The left column contains the photo upload area — supporting drag-and-drop, file browsing and clipboard paste — and the

vehicle and policy form: car segment, city tier, vehicle age, engine capacity, IDV, zero-depreciation add-on and voluntary deductible. The right column shows the annotated photograph and the estimate after the user clicks *Estimate claim*. Five sample photos let a new user try the system with one click. Models are loaded once and cached, so each request only runs inference.

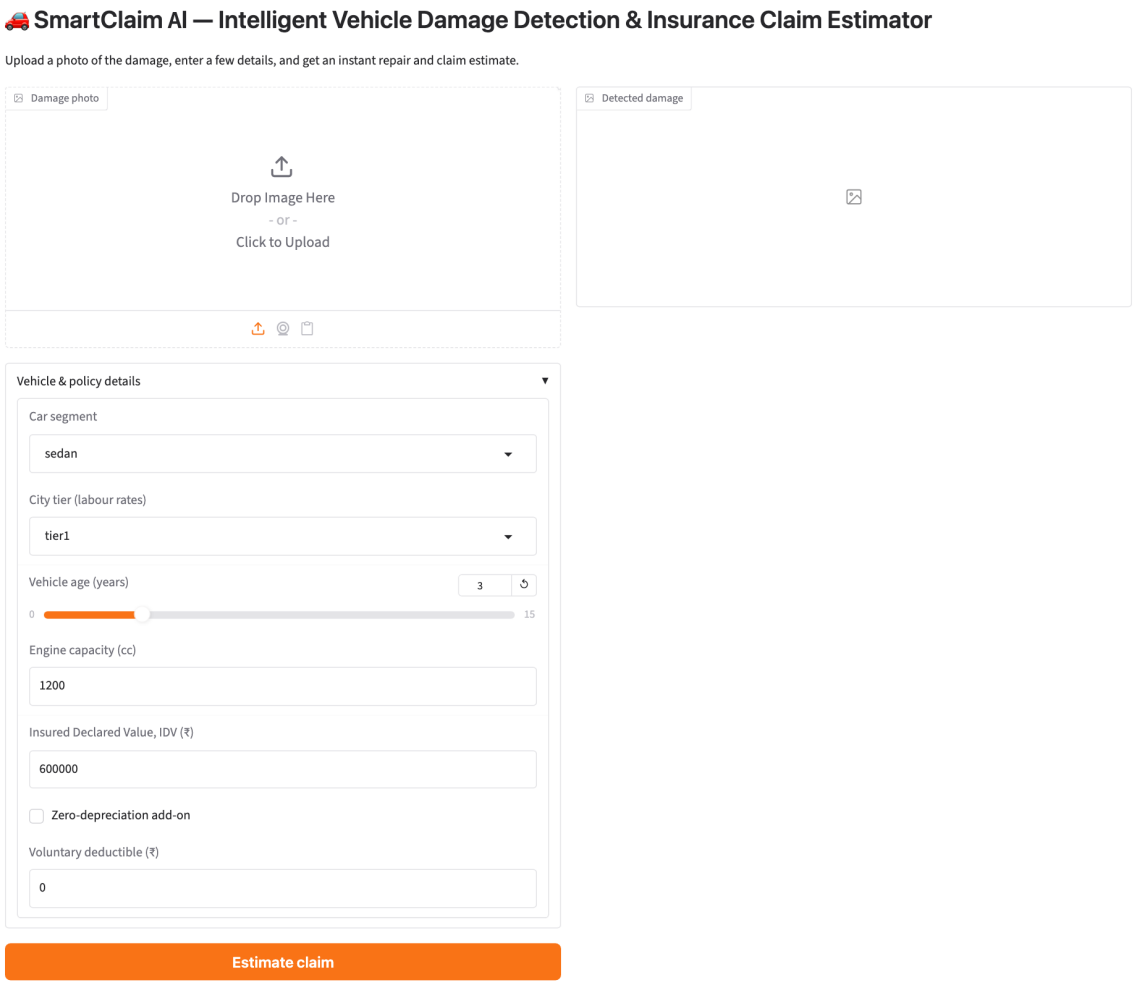


Figure 6.4: SmartClaim AI web application — input screen

6.7 Testing

The estimator is covered by 32 automated pytest tests that check the depreciation schedule for every age band, deductibles, that the low estimate never exceeds the high estimate, that crushed damage costs more than breakage, segment scaling, settlement arithmetic, zero-depreciation cover, total-loss settlement, low-confidence flags, damage-to-part overrides, front/rear correction and the handling of cosmetic damage on unpainted parts. All tests pass.

7. Results

7.1 Location and Severity Classification

Table 7.1 and Figure 7.1 compare the five classifiers on the 575 held-out validation images. Transfer learning clearly beats training from scratch, and the YOLOv8n-cls model is the most accurate while also being the smallest, so it was selected for the final system.

Table 7.1: Validation accuracy of the five classifiers

Model	Validation accuracy
Baseline CNN	48.70%
CNN + Regularisation	46.61%
EfficientNet-B0 (frozen backbone)	60.70%
ResNet50 (layer4 fine-tuned)	73.91% (best epoch 77.57%)
YOLOv8n-cls (selected)	**80.35%**

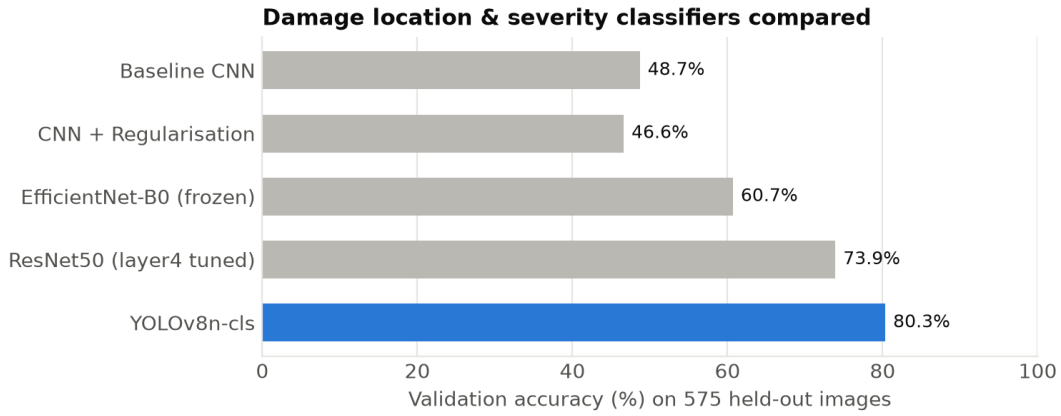


Figure 7.1: Validation accuracy of the five classifiers

Figure 7.2 shows that YOLOv8n-cls improved steadily over ten epochs, from 61.2% after the first epoch to 80.3%, with validation loss flattening near the end.

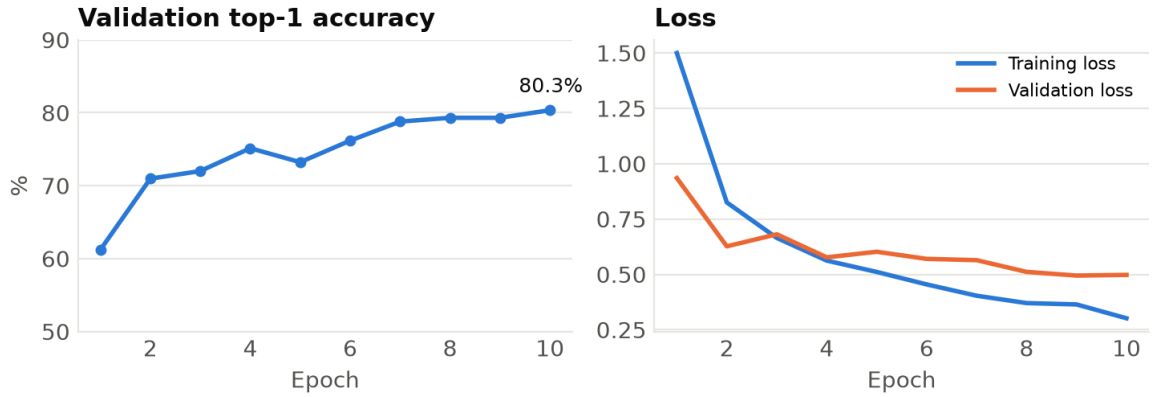


Figure 7.2: YOLOv8n-cls training: validation accuracy and loss per epoch

The per-class results (Table 7.2, Figure 7.3, Figure 7.4) show that front damage is recognised most reliably (F1 0.80–0.89), while rear crushed and rear normal photos are harder (F1 0.65–0.69), largely because those classes have fewer training images.

Table 7.2: Per-class precision, recall and F1-score of YOLOv8n-cls

Class	Precision	Recall	F1-score	Support
F_Breakage	0.88	0.90	0.89	125
F_Crushed	0.76	0.84	0.80	100
F_Normal	0.92	0.83	0.87	125
R_Breakage	0.80	0.81	0.81	75
R_Crushed	0.63	0.68	0.65	75
R_Normal	0.73	0.65	0.69	75
Weighted average	**0.81**	**0.80**	**0.80**	**575**

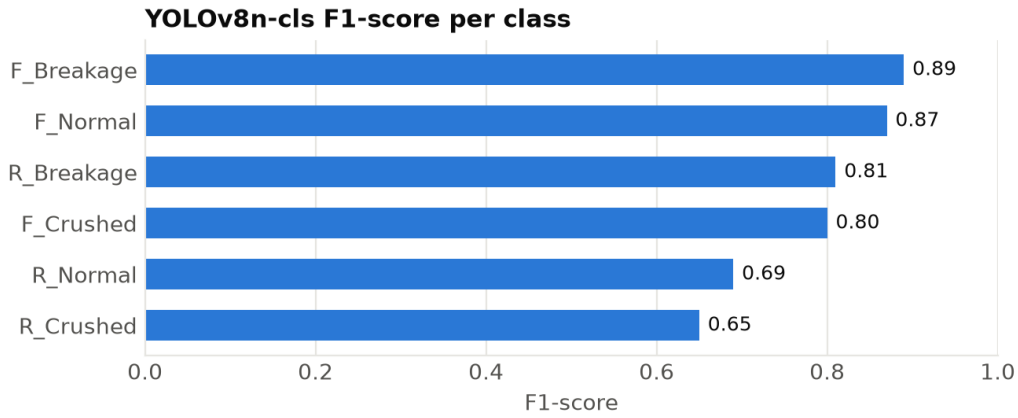


Figure 7.3: F1-score per class for the selected classifier

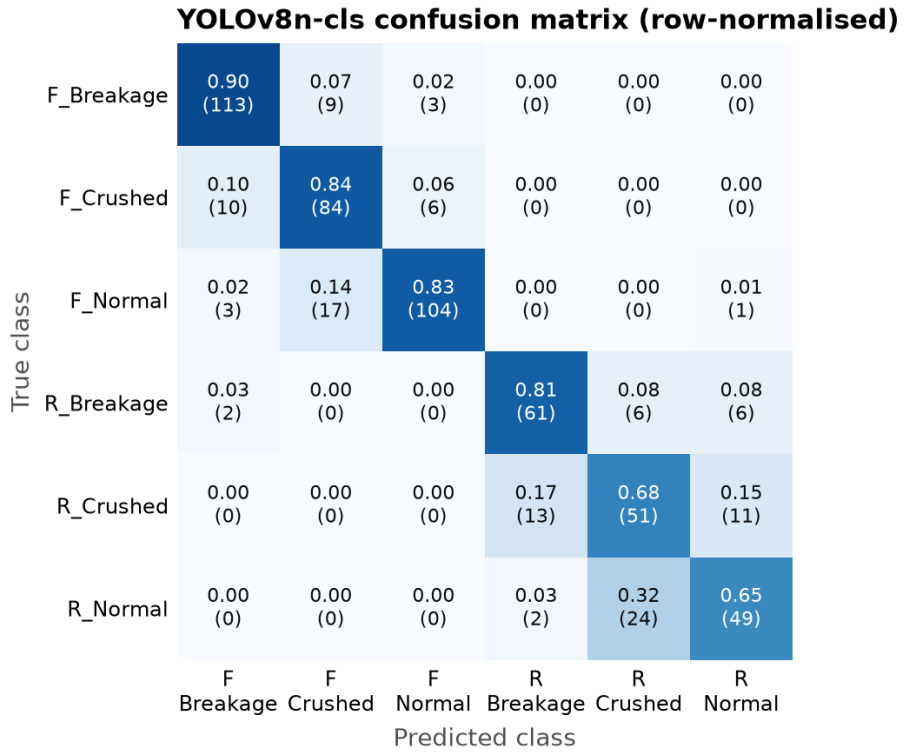


Figure 7.4: Confusion matrix of YOLOv8n-cls on the 575 validation images

7.2 Part and Damage Segmentation

Table 7.3 summarises the segmentation models and Figure 7.5 shows their learning curves. The car-part model learns quickly, reaching a mask mAP@0.5 of 0.81 after six epochs with precision 0.81 and recall 0.75.

Table 7.3: Segmentation results on the validation split

Model	Box mAP@0.5	Mask mAP@0.5	Mask mAP@0.5:0.95	Precision	Recall
Car parts (21 classes)	0.828	0.812	0.539	0.809	0.751
Damage (8 classes)	0.171	0.154	0.083	0.405	0.184

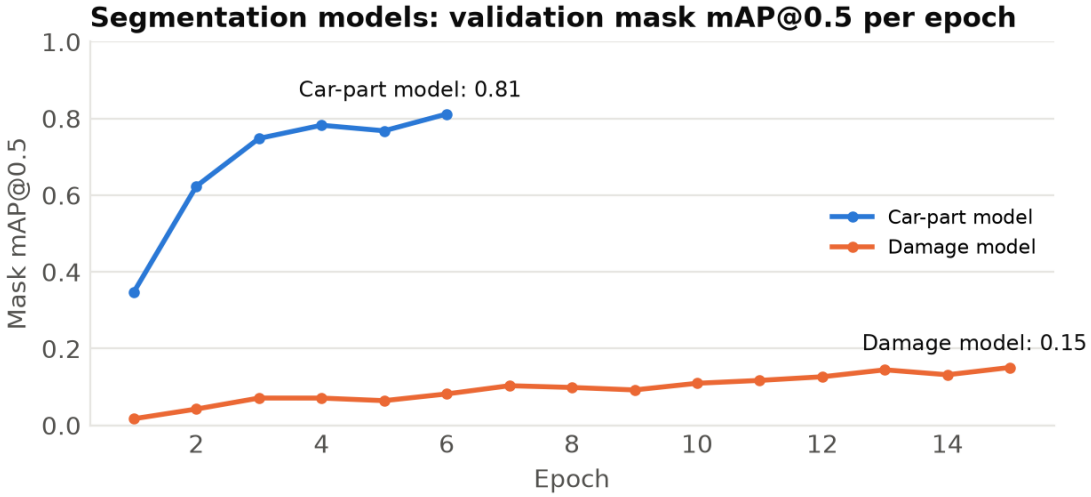


Figure 7.5: Validation mask mAP@0.5 per epoch for both segmentation models

Per-class results for the part model (Figure 7.6) are strong for large, distinctive parts such as wheels, windows, doors and bumpers, which are exactly the parts that dominate repair cost. Figure 7.7 shows its predictions on a validation batch.

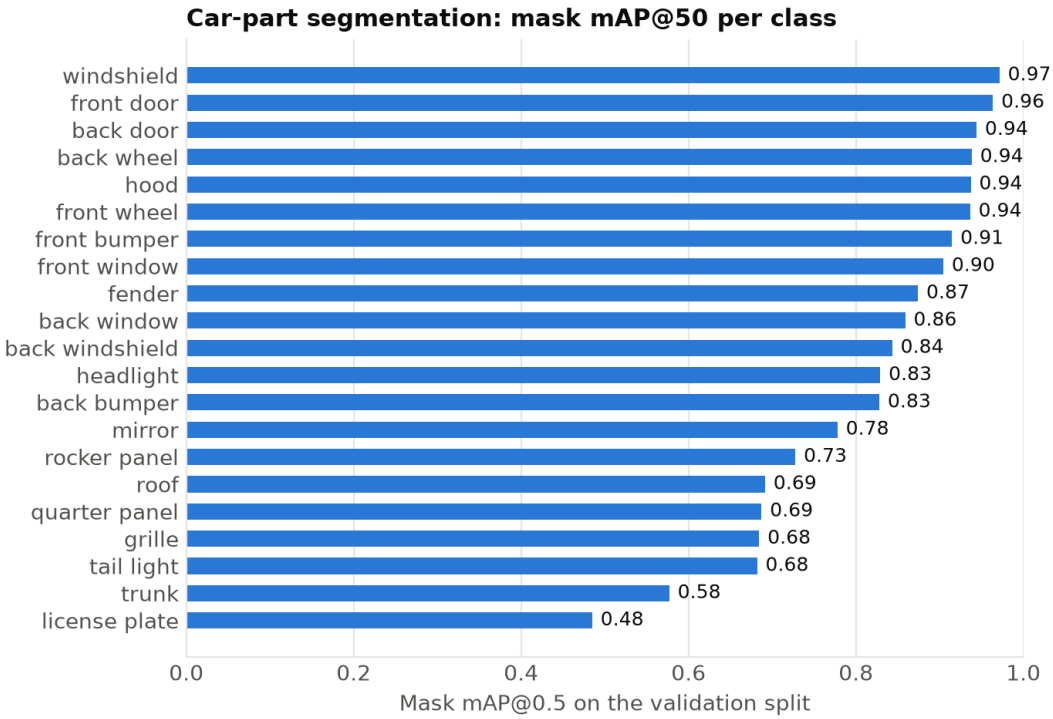


Figure 7.6: Car-part segmentation: mask mAP@0.5 for each of the 21 parts

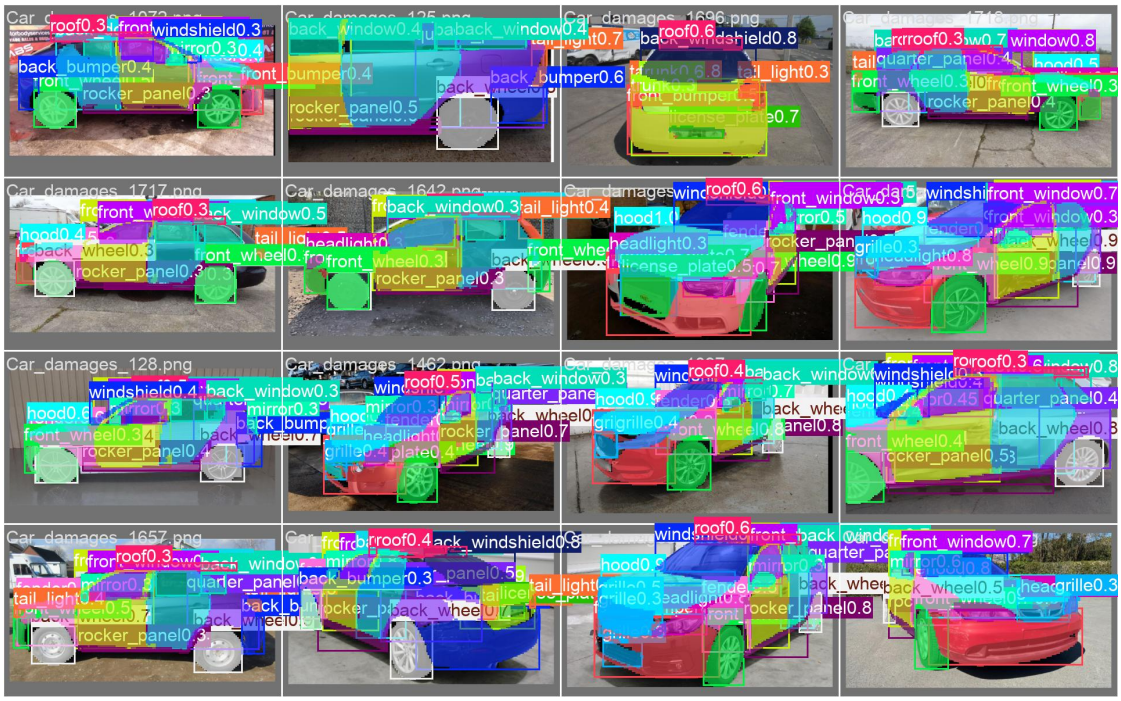


Figure 7.7: Car-part model predictions on validation images

Damage segmentation is a harder task: damage regions are small, irregular and visually similar to reflections and dirt. Figure 7.8 shows that the model is most reliable for the damage types that drive cost — **cracked (0.44), missing part (0.35) and broken part (0.25)** — and weakest for thin surface damage such as scratches and flaking. This is why the estimator combines the damage model with the classifier: the classifier's severity scenario defines the high estimate, so a missed scratch never hides a crushed panel, while the damage model adds part-level precision whenever it detects something.

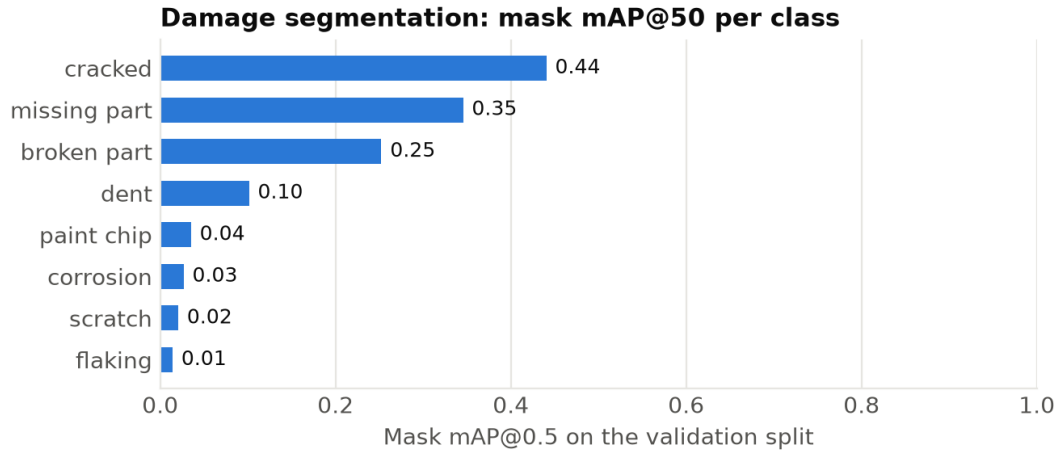


Figure 7.8: Damage segmentation: mask mAP@0.5 for each damage type



Figure 7.9: Damage model predictions on validation images

7.3 End-to-End Claim Estimates

Figure 7.10 and Figure 7.11 show the complete output for a held-out photograph of a sedan with front damage (vehicle age 3 years, 1,200 cc, IDV ₹6,00,000, tier-1 city). The classifier labels the photo *front, crushed*; the damage model finds a broken headlamp, a dented fender and a missing section of the front bumper. The low estimate prices exactly these three parts, ₹26,450, of which the insurer pays ₹17,564. The high estimate adds the grille, bonnet, radiator and radiator support that are typically damaged in a crushed front, giving ₹74,729 with ₹58,397 payable by the insurer.

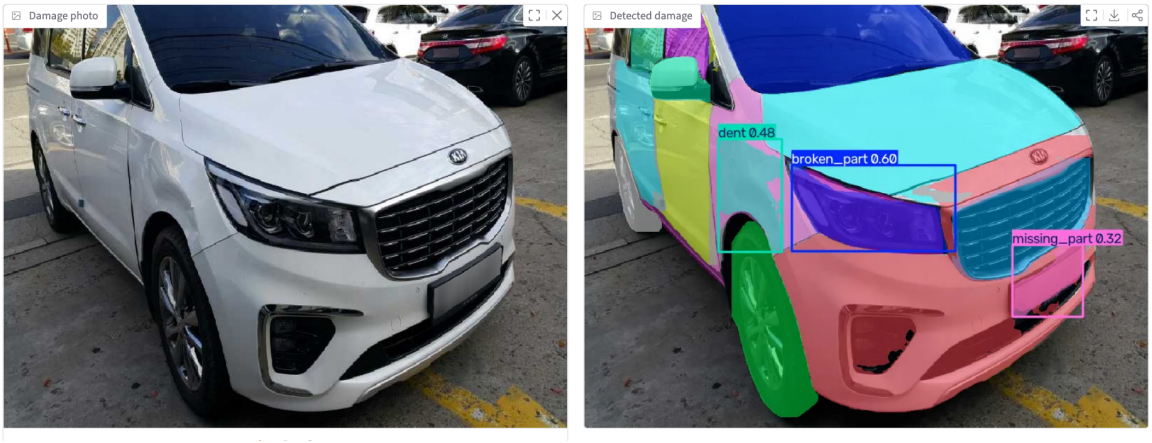


Figure 7.10: Front-damage example: uploaded photo and detections

Damage: Front — Crushed · confidence 56%

Damage found

Damage	On part	Confidence
broken part	headlamp	60%
dent	fender	48%
missing part	front bumper	32%

Estimated repair: ₹26,450 – ₹74,729

Insurer pays ₹17,564 – ₹58,397, you pay ₹8,885 – ₹16,333

Low estimate

Part	Action	Cost
headlamp	replace	₹6,909
fender	repair	₹6,761
front bumper	replace	₹12,779
Repair bill (incl. GST)		**₹26,450**
– depreciation		₹7,885
– deductible		₹1,000
Insurer pays		**₹17,564**
You pay		**₹8,885**

High estimate

Part	Action	Cost
headlamp	replace	₹6,909
fender	replace	₹11,186
front bumper	replace	₹12,779
grille	replace	₹3,251
bonnet	replace	₹17,175
radiator	replace	₹11,092
radiator support	replace	₹12,337
Repair bill (incl. GST)		**₹74,729**
– depreciation		₹15,333
– deductible		₹1,000
Insurer pays		**₹58,397**
You pay		**₹16,333**

Figure 7.11: Front-damage example: estimates and settlement

Table 7.4 reproduces the calculation for one line item — replacing the front bumper of this sedan — to show that every figure can be traced to the rules in Chapter 4.

Table 7.4: Worked example: replacing the front bumper of a 3-year-old sedan

Component	Calculation	Amount (₹)
Part price	4,500 × 1.35 (sedan)	6,075
Labour	1.5 h × ₹650 (tier-1)	975
Paint	2,800 × 1.35	3,780
GST	18% of 10,830	1,949
Line total		**12,779**
Depreciation — part	6,075 × 1.18 × 50% (plastic)	3,584
Depreciation — paint material	3,780 × 25% × 1.18 × 50%	558

Figure 7.12 and Figure 7.13 show a rear-damage example in which the policy has the zero-depreciation add-on. The model attributes the broken part and scratches to the rear bumper; because depreciation is waived, the customer pays only the ₹1,000 compulsory deductible in both scenarios while the insurer pays ₹11,779–₹15,668.

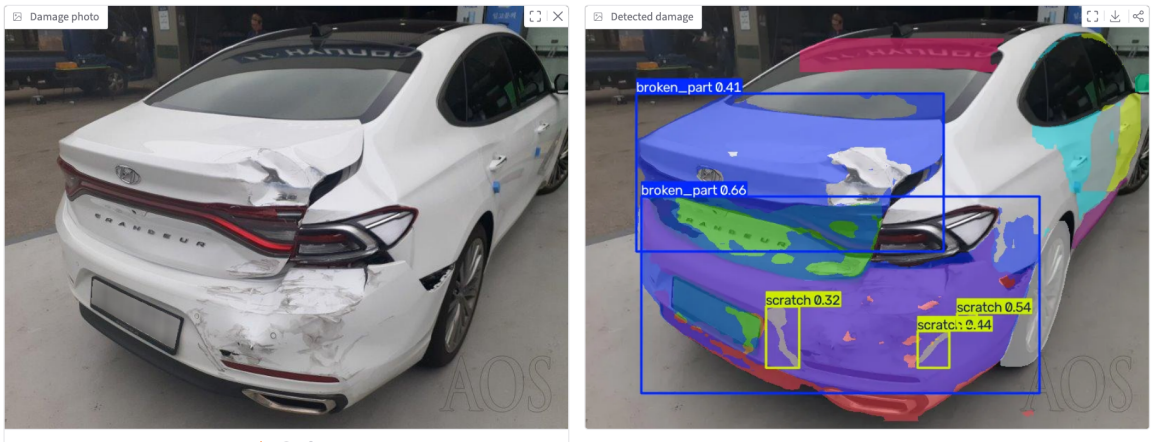


Figure 7.12: Rear-damage example (zero-depreciation cover): detection

Damage: Rear — Breakage · confidence 99%

Damage found

Damage	On part	Confidence
broken part	rear bumper	66%
scratch	rear bumper	54%
scratch	rear bumper	44%
broken part	(by zone)	41%
scratch	rear bumper	32%

Estimated repair: ₹12,779 – ₹16,668

Insurer pays ₹11,779 – ₹15,668, you pay ₹1,000 – ₹1,000

Low estimate

Part	Action	Cost
rear bumper	replace	₹12,779
Repair bill (incl. GST)		**₹12,779**
– depreciation		₹0
– deductible		₹1,000
Insurer pays		**₹11,779**
You pay		**₹1,000**

High estimate

Part	Action	Cost
rear bumper	replace	₹12,779
tail lamp	replace	₹3,888
Repair bill (incl. GST)		**₹16,668**
– depreciation		₹0
– deductible		₹1,000
Insurer pays		**₹15,668**
You pay		**₹1,000**

Figure 7.13: Rear-damage example: estimates and settlement

7.4 Performance

On the Apple M4 GPU the complete pipeline — classification, part segmentation, damage segmentation, attribution, pricing and rendering of the annotated image — returns a result in about **1.9 seconds** per photograph through the web application. The classifier weights occupy 2.8 MB (5.5 MB as ONNX), and the two segmentation models 6–28 MB, so the whole system fits comfortably on a laptop.

8. Conclusion

SmartClaim AI demonstrates that the first assessment of a motor insurance claim can be produced automatically, in seconds, from a single photograph and a handful of policy details. The project delivered a complete, working pipeline:

- A damage location and severity classifier selected from five architectures, with YOLOv8n-cls reaching 80.35% validation accuracy — 31.7 points above a custom CNN and 6.4 points above a fine-tuned ResNet50.
- A car-part segmentation model covering 21 exterior parts with a mask mAP@0.5 of 0.81, and a damage segmentation model for eight damage types that is most reliable on the cost-driving classes of broken, missing and cracked parts.
- A damage-to-part attribution method that converts pixel masks into named, repairable parts, with automatic correction of front/rear confusions using the classifier.
- A transparent pricing engine and an implementation of the IRDAI depreciation schedule, deductibles, zero-depreciation cover and total-loss rule that split each bill into the insurer's and the customer's share.
- A web application that presents the annotated photo, a low-to-high estimate and an itemised breakdown, backed by 32 automated tests.

An important finding of the project is that historical claim amounts in public datasets do not reflect visible damage, so a reliable estimator has to reason about parts, prices and policy rules rather than learn a single number from claim records. By keeping the vision models and the pricing rules separate, SmartClaim AI produces estimates that are explainable line by line — a property that matters to customers, insurers and regulators alike.

9. Future Scope

- **Larger damage datasets and longer training:** training the damage model on larger annotated collections and for more epochs, and using the small or medium YOLO11 backbones, to improve detection of scratches, paint chips and dents.
- **Multi-photo assessment:** combining several photos of the same vehicle (front, rear, both sides, close-ups) into one consolidated estimate and covering side damage to doors and panels.
- **Live price integration:** replacing the static part catalogue with OEM and workshop price lists per make and model, and city-specific labour rates.
- **Fraud screening:** flagging re-used, edited or re-photographed images and inconsistencies between the photo and the reported incident, building on the claims dataset analysed in this project.
- **Mobile application and claim workflow:** a mobile app that guides the user to take good photos, and integration with insurer claim-registration systems so the estimate travels with the claim.
- **Explainability:** saliency maps for the classifier and confidence-weighted ranges that widen when the models are uncertain.

References

[1] K. Patil, M. Kulkarni, A. Sriraman and S. Karande, “Deep learning based car damage classification,” in *Proc. 16th IEEE Int. Conf. on Machine Learning and Applications (ICMLA)*, 2017, pp. 50–54, doi: 10.1109/ICMLA.2017.0-179.

[2] K. He, X. Zhang, S. Ren and J. Sun, “Deep residual learning for image recognition,” in *Proc. IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.

[3] M. Tan and Q. V. Le, “EfficientNet: Rethinking model scaling for convolutional neural networks,” in *Proc. 36th Int. Conf. on Machine Learning (ICML)*, 2019, pp. 6105–6114.

[4] J. Redmon, S. Divvala, R. Girshick and A. Farhadi, “You only look once: Unified, real-time object detection,” in *Proc. IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788.

[5] K. He, G. Gkioxari, P. Dollár and R. Girshick, “Mask R-CNN,” in *Proc. IEEE Int. Conf. on Computer Vision (ICCV)*, 2017, pp. 2961–2969.

[6] U. Waqas, N. Akram et al., “Vehicle damage classification and fraudulent image detection including Moiré effect using deep learning,” in *Proc. IEEE Canadian Conf. on Electrical and Computer Engineering (CCECE)*, 2020, doi: 10.1109/CCECE47787.2020.9255806.

[7] X. Wang, W. Li and Z. Wu, “CarDD: A new dataset for vision-based car damage detection,” *IEEE Transactions on Intelligent Transportation Systems*, 2023, doi: 10.1109/TITS.2023.3258480.

[8] G. Jocher, J. Qiu and A. Chaurasia, *Ultralytics YOLO*, software, 2023. [Online]. Available: https://github.com/ultralytics/ultralytics

[9] A. Paszke et al., “PyTorch: An imperative style, high-performance deep learning library,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[10] A. Abid, A. Abdalla, A. Abid, D. Khan, A. Alfozan and J. Zou, “Gradio: Hassle-free sharing and testing of ML models in the wild,” arXiv:1906.02569, 2019.

[11] Insurance Regulatory and Development Authority of India, *India Motor Tariff — General Regulations on depreciation, compulsory deductibles and total loss.* [Online]. Available: https://irdai.gov.in

[12] *Car Parts and Damages Polygon Dataset*, MIT licence, Hugging Face. [Online]. Available: https://huggingface.co/datasets/DrBimmer/car-parts-and-damage-dataset

[13] *Auto insurance claims dataset (insurance_claims.csv)*, 1,000 records, 40 attributes, Kaggle.